

Chapter 8: Corrective RAG Pattern (Self-Correcting Retrieval and Adaptive Knowledge Access)

Part II: Agentic Architecture Patterns

Book: Agentic AI & RAG Systems Handbook

Version: 1.0.0

Chapter ID: 08

File Name: chapter-08-corrective-rag-pattern.md

Chapter Overview

Traditional Retrieval-Augmented Generation (RAG) assumes retrieval succeeds on the first attempt.

Production reality is very different.

Failures occur because of:

- Poor queries
- Missing context
- Ambiguous terminology
- Acronyms
- Poor chunking
- Outdated indexes
- Metadata filtering issues
- Low-quality embeddings
- Incomplete retrieval

Corrective RAG (CRAG) introduces a self-correction mechanism.

Instead of:

1	Question
2	↓
3	Retrieve
4	↓
5	Generate
6	↓
7	Answer

Corrective RAG becomes:

1	Question
2	↓
3	Retrieve
4	↓
5	Evaluate
6	↓
7	Correct
8	↓
9	Retrieve Again
10	↓
11	Generate
12	↓
13	Validate
14	↓
15	Answer

The result is a significantly more reliable enterprise retrieval architecture.

Learning Objectives

After completing this chapter you should be able to:

- Understand Corrective RAG architecture
 - Design retrieval evaluation pipelines
 - Build adaptive query rewriting systems
 - Implement retrieval grading frameworks
 - Improve precision and recall
 - Reduce hallucinations
 - Integrate Corrective RAG with LangGraph
 - Measure retrieval quality
 - Design enterprise-grade retrieval systems
-

Prerequisites

Required:

- Chapter 1: Generative AI Foundations
- Chapter 2: Agentic AI Fundamentals
- Chapter 4: ReAct Pattern

- Chapter 7: Reflection Pattern

Recommended:

- Future Chapter 31: RAG Fundamentals
 - Future Chapter 34: Hybrid Search Architecture
-

1. Introduction

Most RAG failures occur before generation.

Example:

Question:

1 What customer data controls are required by policy?

Retriever returns:

1 Customer onboarding procedures

instead of:

1 Data protection policies

Even the best model cannot fix:

1 Wrong Context

Corrective RAG recognizes this reality.

A core enterprise principle:

Bad retrieval cannot be repaired with better generation.

2. Core Concepts

What Is Corrective RAG?

Definition

Corrective RAG is a retrieval architecture that evaluates retrieval quality and performs corrective actions when sufficient evidence is not found.

Core Principle

Traditional RAG:

- | |
|-----------------|
| 1 Retrieve Once |
| 2 Generate Once |

Corrective RAG:

- | |
|------------|
| 1 Retrieve |
| 2 Evaluate |
| 3 Improve |
| 4 Retry |
| 5 Generate |
| 6 Validate |

3. Why Corrective RAG Exists

Enterprise retrieval frequently fails because:

Query Ambiguity

- | |
|--------------------------|
| 1 Show retention policy. |
|--------------------------|

Which policy?

- Email?
 - HR?
 - Finance?
 - Legal?
-

Acronyms

- | |
|------------------------|
| 1 Explain RCA process. |
|------------------------|

Could mean:

- Root Cause Analysis
 - Risk Control Assessment
 - Regional Compliance Audit
-

Missing Context

- | |
|--------------------------|
| 1 Show recent incidents. |
|--------------------------|

Recent for:

- Application?
- Customer?

- Region?
 - Team?
-

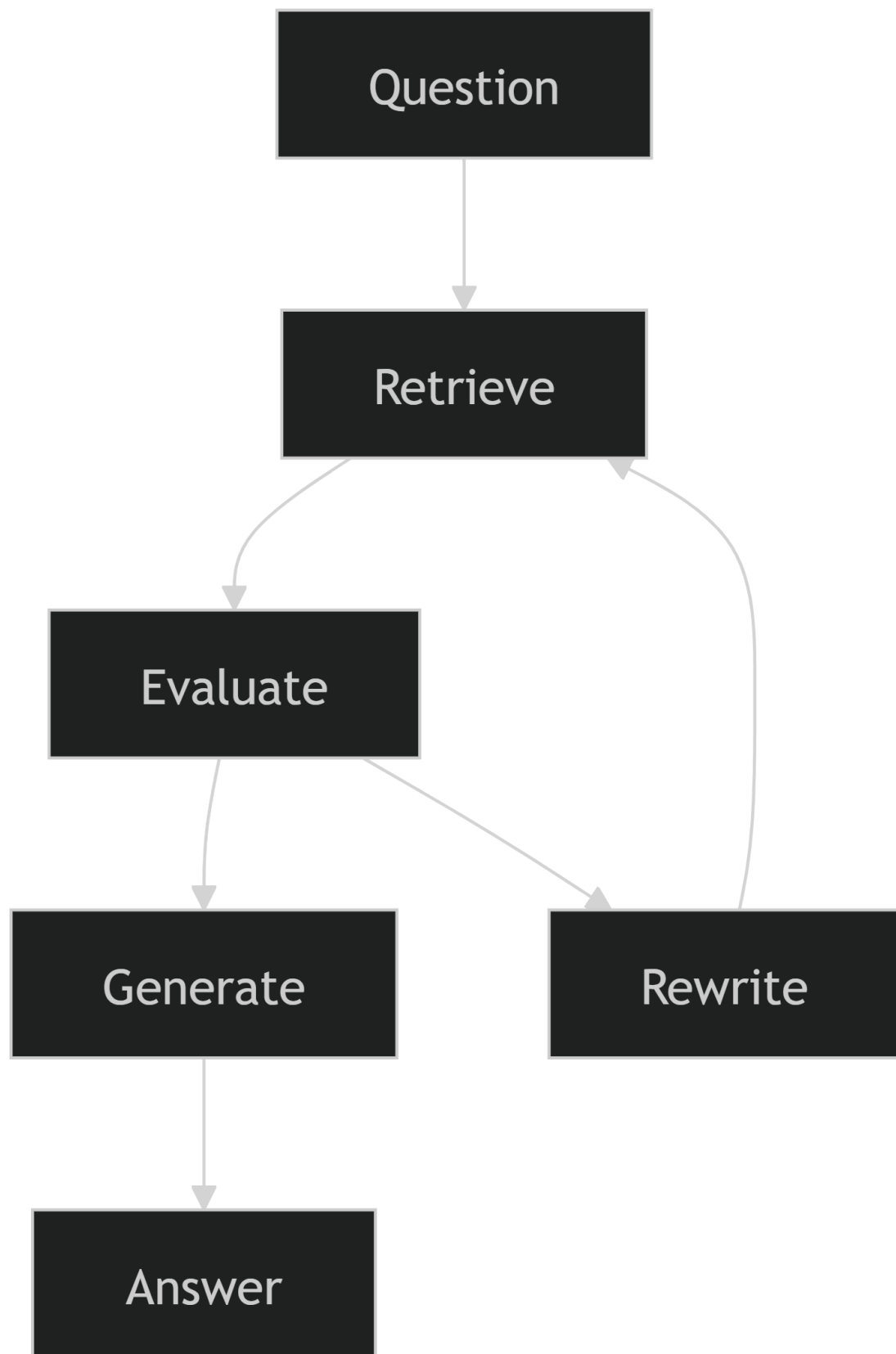
Retrieval Drift

Indexes evolve over time.

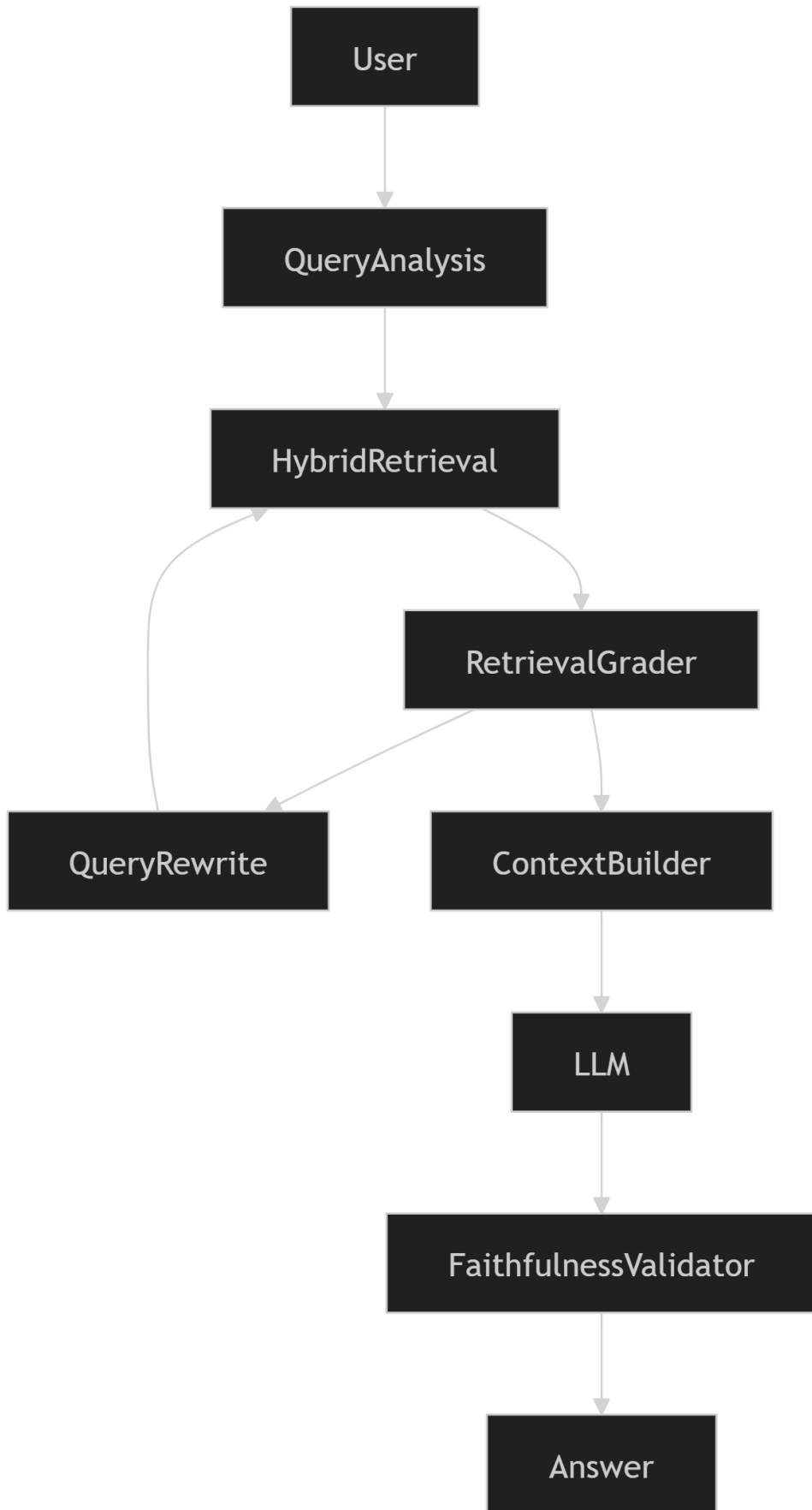
Queries may stop performing well.

4. Corrective RAG Architecture

Basic Architecture



Enterprise Architecture



5. Retrieval Evaluation Layer

The most important part of Corrective RAG.

Question:

1 Did retrieval succeed?

Most RAG systems never ask this.

Evaluation Criteria

Relevance

Did retrieved content match intent?

Sufficiency

Was enough evidence found?

Coverage

Were all aspects addressed?

Freshness

Is information current?

Security

Was authorized content retrieved?

6. Retrieval Grading

Definition

A retrieval grader scores retrieved content.

Example

Question:

1	How do I rotate access keys?
---	------------------------------

Retrieved:

1	VPN configuration
---	-------------------

Score:

1	Low
---	-----

High Quality Retrieval

Question:

1	How do I rotate access keys?
---	------------------------------

Retrieved:

1	IAM key rotation policy
---	-------------------------

Score:

1	High
---	------

7. Query Rewriting

One of the strongest corrective mechanisms.

Original Query

1	RCA process
---	-------------

May fail due to ambiguity.

Rewritten Query

1	Root Cause Analysis incident management process
---	---

Retrieval quality improves dramatically.

Architecture



8. Query Expansion

Instead of rewriting:

expand.

Example:

Query:

1 PCI controls

Expanded:

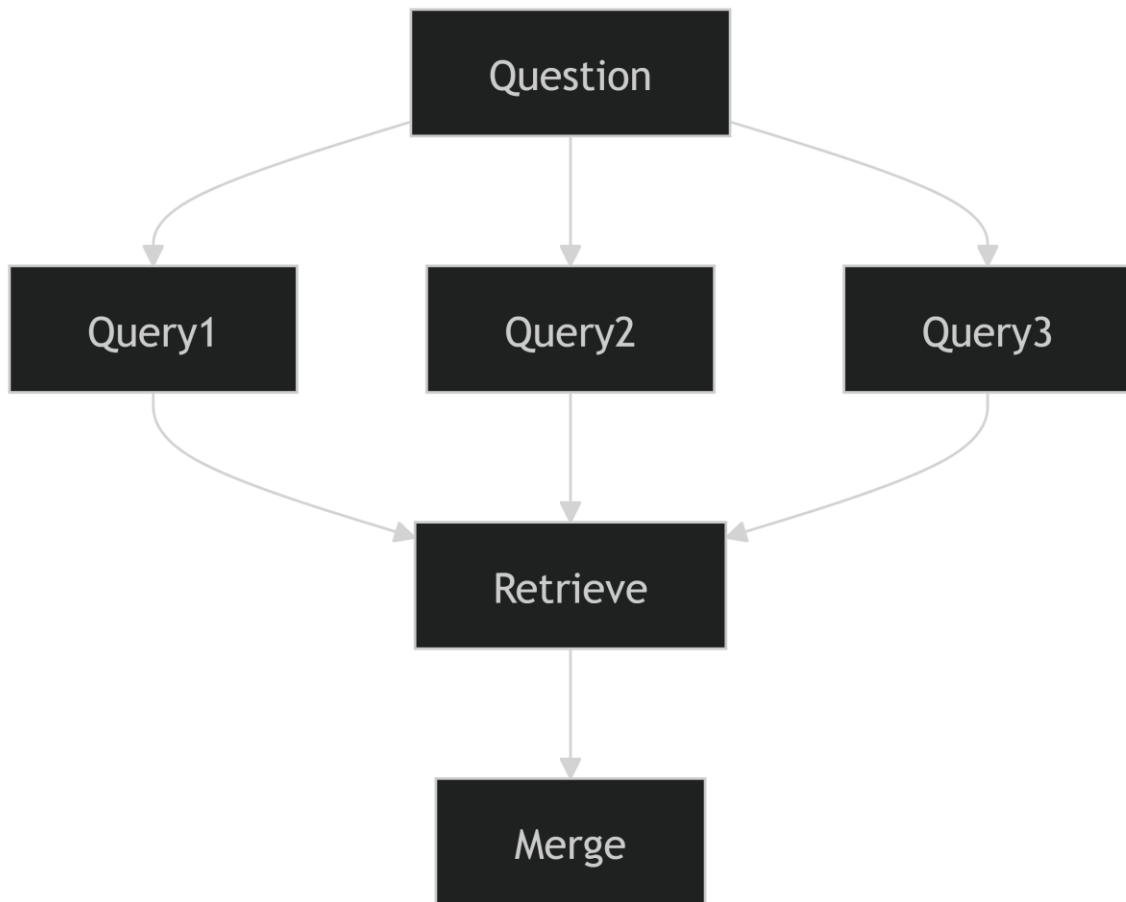
1 PCI DSS
2 Payment Card Industry
3 Compliance Controls
4 Cardholder Data Controls

Benefits:

- Higher recall
 - Better retrieval coverage
-

9. Multi-Query Retrieval

Multiple retrieval paths execute simultaneously.



Example

Original:

1 Cloud security controls

Generated:

1 Cloud governance
2 Cloud compliance
3 Cloud security standards

10. Retrieval Correction Strategies

Strategy 1: Retry Retrieval

Simple.

1 Try again.

Strategy 2: Rewrite Query

Most common.

Strategy 3: Expand Query

Increases recall.

Strategy 4: Multi-Retriever

Use:

1	BM25
2	+
3	Vector Search
4	+
5	Graph Search

Strategy 5: Human Escalation

When uncertainty remains.

11. Retrieval Models

Rules-Based

1	if score < 0.60:
2	retry()

Embedding Similarity

Measure retrieval relevance.

Cross Encoder Scoring

Higher quality.

More expensive.

LLM-Based Evaluation

Question:

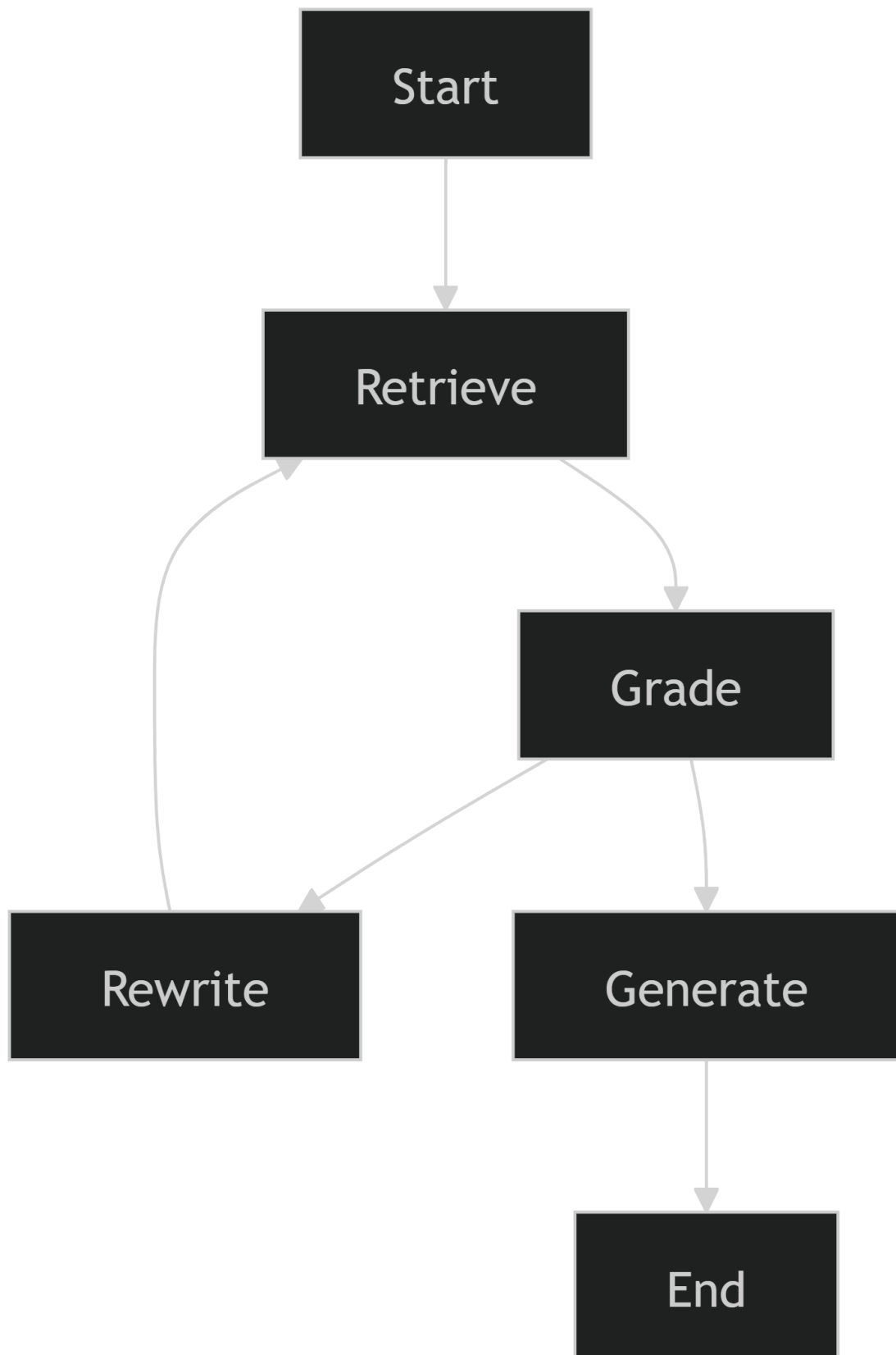
1 Does retrieved content answer the question?

LLM scores retrieval quality.

12. LangGraph Corrective RAG

Corrective RAG maps naturally to LangGraph.

Graph



State

```
1 class CRAGState(TypedDict):
2     question: str
3     documents: list
4     grade: float
5     answer: str
```

Retrieval Node

```
1 def retrieve(state):
2     docs = retriever.search(
3         state["question"]
4     )
5
6     return {
7         "documents": docs
8     }
```

Grader Node

```
1 def grade(state):
2
3     score = evaluator(
4         state["documents"]
5     )
6
7     return {
8         "grade": score
9     }
```

Rewrite Node

```
1 def rewrite(state):
2
3     better_query = rewrite_query(
4         state["question"]
5     )
6
```

```
7     return {  
8         "question": better_query  
9     }
```

13. Corrective RAG + ReAct

Powerful combination.

```
1 Reason  
2 ↓  
3 Retrieve  
4 ↓  
5 Evaluate  
6 ↓  
7 Rewrite  
8 ↓  
9 Retrieve  
10 ↓  
11 Reason
```

Produces strong enterprise reasoning systems.

14. Corrective RAG + Reflection

Another high-value architecture.

```
1 Retrieve  
2 ↓  
3 Generate  
4 ↓  
5 Reflect  
6 ↓  
7 Evidence Check  
8 ↓  
9 Improve
```

15. Corrective RAG + GraphRAG

Graph retrieval can act as a corrective path.

1	Vector Retrieval Fails
2	↓
3	Graph Retrieval
4	↓
5	Evidence Recovery

Especially effective for:

- Compliance
 - RCA
 - SOP systems
-

16. Enterprise Compliance Example

Question:

1	Which PCI controls govern data retention?
---	---

Initial retrieval:

1	Data retention procedures
---	---------------------------

Problem:

1	Missing PCI references
---	------------------------

Correction:

1	Retrieve PCI DSS controls
---	---------------------------

New retrieval:

1	PCI control mappings
---	----------------------

Final answer becomes complete.

17. Enterprise RCA Example

Question:

1	What caused this outage?
---	--------------------------

First retrieval:

1	Incident summary
---	------------------

Incomplete.

Correction:

- | |
|--|
| <ol style="list-style-type: none">1 Search previous incidents2 Search monitoring events3 Search deployment history |
|--|

Significantly improves investigation quality.

18. Adaptive RAG

Corrective RAG evolves into Adaptive RAG.

Adaptive systems dynamically decide:

- | |
|---|
| <ol style="list-style-type: none">1 Search?2 Rewrite?3 Graph Retrieval?4 Human Escalation? |
|---|

based on retrieval quality.

19. Scalability

Corrective RAG increases:

- Retrieval count
- Token usage
- Infrastructure load

Architecture:

- | |
|--|
| <ol style="list-style-type: none">1 Better Answers2 but3 Higher Cost |
|--|

Optimization

Only invoke correction when:

- | |
|--|
| <ol style="list-style-type: none">1 Confidence Low |
|--|

20. Reliability

Benefits:

- Higher recall
- Better grounding
- Fewer hallucinations

Costs:

- Higher latency
 - More infrastructure
-

21. Security

Corrective RAG must preserve:

- ACL filtering
- Security trimming
- Metadata filtering

Bad architecture:

1	Retry
2	↓
3	Bypass Security

Never acceptable.

22. Observability

Track:

Retrieval Quality

1	Average Retrieval Score
---	-------------------------

Correction Rate

1	Corrections Per Answer
---	------------------------

Rewrite Frequency

1	Query Rewrite Count
---	---------------------

Escalation Rate

1 Failed Retrievals

23. Failure Modes

Failure 1

Infinite Retrieval Loops

Failure 2

Poor Query Rewrites

Failure 3

Bad Graders

Failure 4

Incorrect Confidence Scores

Failure 5

Cost Explosion

24. Common Mistakes

Mistake 1

Trusting retrieval blindly.

Mistake 2

No retrieval evaluation.

Mistake 3

Always rewriting.

Mistake 4

Ignoring retrieval metrics.

Mistake 5

No escalation path.

25. Anti-Patterns

Retrieval Once Architecture

- | | |
|---|----------|
| 1 | Retrieve |
| 2 | ↓ |
| 3 | Generate |

No validation.

Rewrite Everything

Unnecessary cost.

No Evidence Validation

High hallucination risk.

Correction Without Limits

Leads to runaway execution.

26. Trade-Offs

Benefit	Cost
Better retrieval	More latency
Better grounding	More infrastructure
Lower hallucination rate	More tokens
Better quality	More complexity

27. Architecture Decision Matrix

Scenario	Corrective RAG Fit
Enterprise Search	Excellent
Compliance Systems	Excellent
Legal Systems	Excellent
Incident Analysis	Excellent
Knowledge Management	Excellent
FAQ Bot	Moderate
Simple Search	Limited

28. Real-World Use Cases

Enterprise Knowledge Search

Correct retrieval validation.

Compliance Assistants

Evidence verification.

IT Operations

Incident research.

SOP Platforms

Policy retrieval.

Agentic Operations Platforms

Adaptive retrieval workflows.

29. Interview Questions

Q1

What is Corrective RAG?

Q2

Why does traditional RAG fail?

Q3

What role does retrieval grading play?

Q4

How does query rewriting help?

Q5

How do you evaluate retrieval quality?

Q6

How does Corrective RAG reduce hallucinations?

Q7

What are major production risks?

Q8

How would you implement Corrective RAG in LangGraph?

30. Interview Answers

Q1

A self-correcting retrieval architecture that evaluates retrieval quality and performs corrective actions before generation.

Q2

Because retrieval often misses relevant, sufficient, or current information.

Q3

It determines whether retrieved evidence is good enough.

Q4

It improves recall, precision, and ambiguity handling.

Q5

Through recall, precision, relevance, coverage, and sufficiency metrics.

Q6

By improving evidence quality prior to generation.

Q7

Cost growth, retry loops, and grading failures.

Q8

Using retrieval, grading, rewrite, and generation nodes connected via conditional routing.

31. Architecture Review Checklist

Corrective RAG Readiness

- Retrieval evaluation implemented
 - Query rewriting implemented
 - Rewrite limits configured
 - Escalation path defined
 - Retrieval metrics collected
 - Security controls preserved
 - Observability implemented
 - Cost controls configured
 - LangGraph workflow tested
-

32. Practical Exercises

1. Design a Corrective RAG architecture for compliance search.
 2. Build retrieval grading logic.
 3. Implement query rewriting in LangGraph.
 4. Compare Corrective RAG and traditional RAG.
 5. Design a retrieval evaluation framework.
-

33. Key Takeaways

- Retrieval quality determines answer quality.
- Corrective RAG evaluates retrieval before generation.
- Query rewriting improves retrieval precision.
- Retrieval grading is a critical capability.
- Corrective RAG significantly reduces hallucinations.
- LangGraph is a natural platform for Corrective RAG.
- Corrective RAG combines well with ReAct and Reflection.
- Adaptive retrieval is a core enterprise pattern.

- Security controls must remain intact throughout correction.
 - Corrective RAG is one of the most important production RAG architectures.
-

34. References

- Corrective Retrieval-Augmented Generation (CRAG)
 - ReAct Paper
 - LangGraph Documentation
 - Azure AI Search Documentation
 - OpenSearch Documentation
 - Enterprise Retrieval Evaluation Frameworks
 - GraphRAG Research
-

Chapter Status: Completed

Markdown Ready: Yes

PDF Ready: Yes (pending build pipeline)

Next Chapter: Chapter 9 – Human-in-the-Loop (HITL) Pattern and Human-Centric Decision Systems